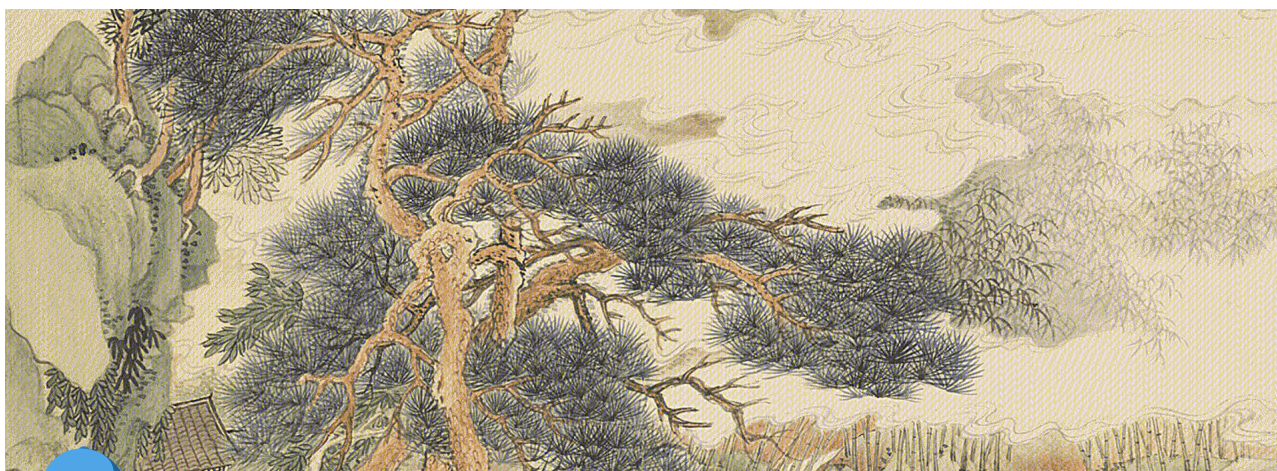




2024 年编译实验 · 编译器设计文档

类别

计算机课程 CT

标签

已完成

新增属性

1 参考编译器介绍



本设计参考了课程组提供的 **Pascal** 编译器的设计模式，现对 **Pascal** 主要源码进行介绍。

1.1 总体结构

该编译器总体结构分为典型的词法分析、语法分析、语义分析、代码生成以及错误处理五个部分。

1.2 主要接口设计

1.2.1 读取下一个字符

▼ nextch

Pascal

```
procedure nextch;
begin
  if cc = ll { * 如果读到行尾 *}
  then begin
    if eof( psin ) { * 要读入的文件已经读完 *}
    then begin
      writeln( psout ); { * 输出空行 *}
      writeln( psout, 'program incomplete' ); { * 输出'program incomplete' *}
    end
  end
  errormsg; { * 输出错误信息 *}
end;
```

```

        exit;
    end;
    if errpos <> 0 { * 有错误出现需要处理 * }
    then begin
        if skipflag then endskip;
        writeln( psout );
        errpos := 0
    end;
    write( psout, lc: 5, ' '); { * 在list文件中输出当前PCODE的行号以及一个空格 * }
    ll := 0; { * 将行长度和字符指针置零 * }
    cc := 0;
    while not eoln( psin ) do
    begin
        ll := ll + 1; { * 统计当前行长度 * }
        read( psin, ch ); { * 读入一个字符 * }
        write( psout, ch ); { * 输出该字符 * }
        line[ll] := ch { * 将ch保存到line中,循环结束line保存到当前行末的所有字符 * }
    end;
    ll := ll + 1;
    readln( psin ); { * 从psin读一行空行,让psin指向下一行代码 * }
    line[ll] := ' '; { * 将行末置为空格 * }
    writeln( psout );
end;
cc := cc + 1; { * 字符指针后移一位 * }
ch := line[cc]; { * 读取下一个字符 * }
end { nextch };

```

1.2.2 错误处理

▼ error

Pascal

```

{ * 打印出错位置和错误码,并将错误编号加入errs中 * }
procedure error( n: integer ); { * n为错误码 * }
begin
    if errpos = 0
    then write ( psout, '****' );
    if cc > errpos
    then begin
        write( psout, ' ': cc-errpos, '^', n:2);
        errpos := cc + 3;
        errs := errs +[n] { * 将错误号加入errs集合 * }
    end
end { error };

```

1.2.3 自定义类型声明处理

▼ typedeclaration

Pascal

```

{ * 处理类型自定义,将自定义的类型信息填入tab * }
procedure typedeclaration;
var tp: types;
    rf, sz, t1 : integer;
begin

```

```

insymbol;
test([ident], blockbegsys, 2 ); { *检查获取到的是不是标识符* }
while sy = ident do
begin
    enter(id, typel);
    t1 := t;
    insymbol;
    if sy = eql
    then insymbol
    else begin
        error(16);
        if sy = becomes
        then insymbol
        end;
    typ( [semicolon, comma, ident] + fsys, tp, rf, sz ); { * 通过调用typ过程获取类型填表所需域
* }

    with tab[t1] do { *将typ的返回值填表* }
    begin
        typ := tp;
        ref := rf;
        adr := sz { * 类型的adr存的是类型所需大小 * }
    end;
    testsemicolon
end

```

1.2.4 变量声明处理

▼ variabledeclaration

Pascal

```

{ * 处理普通变量声明，将变量填入tab * }
procedure variabledeclaration;
var tp : types;
    t0, t1, rf, sz : integer;
begin
    insymbol;
    while sy = ident do { *是标识符* }
    begin
        t0 := t;
        entervariable; { *将变量名录到符号表中* }
        while sy = comma do { *多个变量名逗号分隔* }
        begin
            insymbol;
            entervariable;
        end;
        if sy = colon { *冒号* }
        then insymbol
        else error(5);
        t1 := t;
        typ([semicolon, comma, ident] + fsys, tp, rf, sz ); { *获得类型地址和大小* }
        while t0 < t1 do { *对一行的变量循环填表* }
        begin
            t0 := t0 + 1;
            with tab[t0] do

```

```

        begin
            typ := tp;
            ref := rf;
            lev := level;
            adr := dx;
            normal := true;
            dx := dx + sz
        end
    end;
    testsemicolon
end
end { variabledeclaration };

```

1.2.5 过程和函数声明处理

▼ procdeclaration

Pascal

{* 处理过程与函数声明，将函数名或过程名填入tab *

```

procedure procdeclaration;
var isfun : boolean; {是否是function*}
begin
    isfun := sy = funcsy; {sy是function就是方法，否则是过程 *}
    insymbol;
    if sy <> ident {sy不是标识符则报错 *}
    then begin
        error(2);
        id := '          '
    end;
    if isfun {函数或过程入tab *}
    then enter(id,funktion)
    else enter(id,prozedure);
    tab[t].normal := true;
    insymbol;
    block([semicolon]+fsys, isfun, level+1 ); {递归调用block处理分程序 *}
    if sy = semicolon {读到分号才算过程正常结束，否则报错 *}
    then insymbol
    else error(14);
    emit(32+ord(isfun)) {exit} {生成退出分程序的PCODE *}
end { proceduredeclaration };

```

1.2.6 中间代码生成

▼ emit

Pascal

```

procedure emit( fct: integer ); {fct为操作码,该过程处理无操作数的情况*}
begin
    if lc = cmax {lc为code表的索引变量*}
    then fatal(6);
    code[lc].f := fct;
    lc := lc + 1
end { emit };

procedure emit1( fct, b: integer ); {fct为操作码,该过程处理一个操作数的情况*}

```

```

begin
    if lc = cmax
    then fatal(6);
    with code[lc] do
        begin
            f := fct;
            y := b;
        end;
        lc := lc + 1
    end { emit1 };

procedure emit2( fct, a, b: integer ); {fct为操作码,该过程处理两个操作数的情况*}
begin
    if lc = cmax then fatal(6);
    with code[lc] do
        begin
            f := fct;
            x := a;
            y := b
        end;
        lc := lc + 1;
    end { emit2 };
end { emit2 };

```

1.3 文件组织

Pascal 源码通过将每种处理操作封装成 **procedure** 进行组织，但是没有非常明显的文件层次。审视每个过程的声明顺序，我们可以看出 **Pascal** 总体上遵循了词法分析、语法分析、语义分析等逐步进行的处理顺序。

2 编译器总体设计

2.1 总体结构

本编译器支持从 **SysY** 源代码，转化为 **LLVM IR** 中间代码（该代码也可直接做为目标代码），最后生成 **MIPS32** 目标代码。采用 **Java** 语言实现。且对 **MIPS32** 目标代码的生成进行了优化，参加了竞速排名。

本编译器总体结构为词法分析、语法分析、语义分析、中间代码生成、目标代码生成以及代码优化六个主要部分，并将错误处理单独实现为一个附加部分。

本次实验源代码 **SysY** 的具体文法如下：

▼ 实验文法 Java

```

1  编译单元 CompUnit → {Decl} {FuncDef} MainFuncDef // 1.是否存在Decl 2.是否存在FuncDef
2  声明 Decl → ConstDecl | VarDecl // 覆盖两种声明
3  常量声明 ConstDecl → 'const' BType ConstDef { ',' ConstDef } ';' // 1.花括号内重复0次
4  // 2.花括号内重复多次
5  基本类型 BType → 'int' | 'char' // 覆盖两种数据类型的定义
6  常量定义 ConstDef → Ident [ '[' ConstExp ']' ] '=' ConstInitVal // 包含普通变量、
7  // 一维数组两种情况

```

```

8  常量初值 ConstInitVal → ConstExp | '{' [ ConstExp { ',' ConstExp } ] '}' |
9  StringConst // 1.常表达式初值 2.一维数组初值
10 变量声明 VarDecl → BType VarDef { ',' VarDef } ';' // 1.花括号内重复0次 2.花括号内重复多次
11 变量定义 VarDef → Ident [ '[' ConstExp ']' ] | Ident [ '[' ConstExp ']' ] '='
12 InitVal // 包含普通常量、一维数组定义
13 变量初值 InitVal → Exp | '{' [ Exp { ',' Exp } ] '}' | StringConst // 1.表达式初值
14 //2.一维数组初值
15 函数定义 FuncDef → FuncType Ident '(' [FuncFParams] ')' Block // 1.无形参 2.有形参
16 主函数定义 MainFuncDef → 'int' 'main' '(' ')' Block // 存在main函数
17 函数类型 FuncType → 'void' | 'int' | 'char' // 覆盖三种类型的函数
18 函数形参表 FuncFParams → FuncFParam { ',' FuncFParam } // 1.花括号内重复0次
19 //2.花括号内重复多次
20 函数形参 FuncFParam → BType Ident [ '[' ']' ] // 1.普通变量2.一维数组变量
21 语句块 Block → '{' { BlockItem } '}' // 1.花括号内重复0次 2.花括号内重复多次
22 语句块项 BlockItem → Decl | Stmt // 覆盖两种语句块项
23 语句 Stmt → LVal '=' Exp ';' // 每种类型的语句都要覆盖
24 | [Exp] ';' //有无Exp两种情况
25 | Block
26 | 'if' '(' Cond ')' Stmt [ 'else' Stmt ] // 1.有else 2.无else
27 | 'for' '(' [ForStmt] ';' [Cond] ';' [ForStmt] ')' Stmt // 1. 无缺省, 1种情况
28 //2. ForStmt与Cond中缺省一个, 3种情况 3. ForStmt与Cond中缺省两个, 3种情况
29 //4. ForStmt与Cond全部缺省, 1种情况
30 | 'break' ';' | 'continue' ';'
31 | 'return' [Exp] ';' // 1.有Exp 2.无Exp
32 | LVal '=' 'getint' '(' ')' ';'
33 | LVal '=' 'getchar' '(' ')' ';'
34 | 'printf' '(' StringConst { ',' Exp } ')' ';' // 1.有Exp 2.无Exp
35 语句 ForStmt → LVal '=' Exp // 存在即可
36 表达式 Exp → AddExp // 存在即可
37
38 条件表达式 Cond → LOrExp // 存在即可
39 左值表达式 LVal → Ident [ '[' Exp ']' ] //1.普通变量、常量 2.一维数组
40 基本表达式 PrimaryExp → '(' Exp ')' | LVal | Number | Character // 四种情况均需覆盖
41
42 数值 Number → IntConst // 存在即可
43
44 字符 Character → CharConst
45 一元表达式 UnaryExp → PrimaryExp | Ident '(' [FuncRParams] ')' | UnaryOp UnaryExp
46 // 3种情况均需覆盖,函数调用也需要覆盖FuncRParams的不同情况
47 单目运算符 UnaryOp → '+' | '-' | '!' 注: '!' 仅出现在条件表达式中 // 三种均需覆盖
48 函数实参表 FuncRParams → Exp { ',' Exp } // 1.花括号内重复0次 2.花括号内重复多次
49 //3.Exp需要覆盖数组传参和部分数组传参
50 乘除模表达式 MulExp → UnaryExp | MulExp ('*' | '/' | '%') UnaryExp // 1.UnaryExp
51 //2.* 3./ 4.% 均需覆盖
52 加减表达式 AddExp → MulExp | AddExp ('+' | '-') MulExp // 1.MulExp 2.+ 需覆盖
53 //3.- 需覆盖
54 关系表达式 RelExp → AddExp | RelExp ('<' | '>' | '≤' | '≥') AddExp // 1.AddExp
55 //2.< 3.> 4.≤ 5.≥ 均需覆盖
56 相等性表达式 EqExp → RelExp | EqExp ('==' | '!=') RelExp // 1.RelExp 2.== 3.!= 均需覆盖
57 逻辑与表达式 LAndExp → EqExp | LAndExp '&&' EqExp // 1.EqExp 2.&& 均需覆盖

```

```
58 逻辑或表达式 LOrExp → LAndExp | LOrExp '||' LAndExp // 1.LAndExp 2.|| 均需覆盖
59 常量表达式 ConstExp、AddExp 注：使用的 Token 必须是常量 // 左右即可
```

2.2 主要类设计

2.2.1 错误处理

- `ErrorType`：错误类型枚举。
- `ErrorChecker`：错误检查器，根据错误码找到相应错误，打印错误信息。

2.2.2 前端

- `Extractor`：源代码提取器，将源代码文本文件提取至 `String` 变量中。
- `TokenType`：单词类型。
- `Token`：单词定义。
- `ReservedWordsChecker`：保留字检查器，识别某字符串是否是保留字。
- `Lexer`：词法分析器，将字符流解析成 `Token` 流。
- `Parser`：语法分析器，将 `Token` 流解析成抽象语法树。

2.2.3 中端

- `SymbolTable`：符号表。分别记录变量符号和函数符号。
- `Visitor`：语义分析器，对抽象语法树进行语义错误检查，并构建相应符号表。
- `Builder`：`LLVM IR` 中间表示生成器。

2.2.4 后端

- `Translator`：`MIPS32` 翻译器，将 `LLVM IR` 中间表示逐句翻译为 `MIPS32` 汇编代码。
- `RegisterFile`：寄存器堆，在构建 `MIPS32` 代码过程中调度 32 个寄存器。
- `optimizeX`：优化方法，支持对中间代码和目标代码进行优化。

2.3 文件组织

```
▼ 文件组织 Java
1  src
2  -----Compiler.java // 主程序，调用前端、中端和后端最终生成目标代码
3
4  |---error // 错误处理
5  -----ErrorType.java
6  -----ErrorChecker.java
7
8  |---frontend // 前端，包括词法分析、语法分析
9  |-----preprocess // 预处理
10 |-----lexer // 1.词法分析
```



```

11      -----Token.java
12      -----TokenType.java
13      -----ReservedWordsChecker.java
14      -----Lexer.java
15  |-- ----parser // 2.语法分析
16      |-- ----ast // 语法树结构
17      -----Parser.java
18
19  |--middle //中端，包括语义分析、中间代码生成
20  |-- ----llvm // 语义分析需要的符号表以及中间代码LLVM IR数据结构
21  -----Visitor.java // 3.语义分析
22  -----Builder.java // 4.中间代码生成LLVM IR
23
24  |--backend //后端，包括目标代码生成
25  |-- ----mips // MIPS32数据结构
26  |-- ----utils // MIPS32生成工具
27      -----MacroBuilder.java // 封装MIPS32常用指令序列，仅在内部使用
28      -----VirtualReg.java // 虚拟寄存器
29      -----RegisterFile.java // 寄存器堆
30  -----Translator.java // 5.目标代码生成 6.代码优化（集成在其中实现）

```

3 Compiler · 主程序设计

3.1 功能介绍

为了追求解耦合和简洁性，项目主程序只负责调用编译器各个模块的一系列功能函数，最终生成目标代码。函数的具体实现全部在对应的类中完成。

3.2 主要接口

- `main`：调用一系列主要方法，逐步进行源代码提取、词法分析、语法分析等工作。具体架构如下：

```

1      public static void main(String[] args) throws IOException {
2          // 读取源程序文本文件
3          String content = Extractor.extractSourceCode();
4
5          // 词法分析
6          Lexer.lexicalAnalysis(content);
7
8          // 语法分析
9          Parser.parse();
10         Parser.printResultToFile("parser.txt");
11
12         // 语义分析
13         Visitor.visit(Parser.getCompUnit());
14         Visitor.printResultToFile("symbol.txt");
15         ErrorChecker.printErrorToFile("error.txt");
16
17         if (!ErrorChecker.hasError()) {

```

Java


```

18         // 中间代码生成
19         Builder.build(Parser.getCompUnit());
20         Builder.printResultToFile("llvm_ir.txt");
21         Builder.printResultToFile("llvm_ir.ll");
22
23         // 目标代码生成
24         Translator translator = new Translator(Builder.getModule());
25         translator.translate();
26         translator.printResultToFile("mips.txt");
27     }
28 }

```

4 error 包 · 错误处理设计

4.1 功能介绍

汇总编译过程中词法分析、语法分析以及语义分析阶段的所有要求处理的错误信息，支持按照行号升序输出，要求每行仅一个错误，否则后检查的错误会覆盖先检查的结果。

4.2 主要类

4.2.1 ErrorType

枚举类型，列出所有需要处理的错误类型。

▼ 所有错误类型 C

```

1  // 词法错误
2      // 非法的符号，特指单个&或者|
3      ILLEGAL_SYMBOL("a"),
4  // 语法错误
5      // 缺少分号
6      MISSING_SEMICOLON("i"),
7      // 缺少右小括号
8      MISSING_RPARENT("j"),
9      // 缺少右中括号
10     MISSING_RBRACK("k");
11 // 语义错误
12     // 重定义
13     REDEFINITION("b"),
14     // 未定义
15     UNDEFINED("c"),
16     // 函数参数个数不匹配
17     PARAMS_NUM_MISMATCH("d"),
18     // 函数参数类型不匹配
19     PARAM_TYPE_MISMATCH("e"),
20     // 多余return语句
21     REDUNDANT_RETURN("f"),
22     // 缺少return语句
23

```

```

24     MISSING_RETURN("g"),
25     // 改变常量的值
26     CHANGE_CONST("h"),
27     // printf表达式与格式字符数量不匹配
28     PRINTF_MISMATCH("l"),
29     // 非循环块内break或continue

```

4.2.2 `ErrorChecker`

存储有 `errorList`，用 `TreeMap` 组织，以行号为 `key`，本行错误信息为 `value`，存储整个编译过程的错误信息。并且提供输出方法。

5 前端设计



前端的主要任务是：接受源代码文件作为输入，对其进行词法分析、语法分析得到抽象语法树。

5.1 `Preprocessor` 预处理

5.1.1 功能介绍

预处理的功能较为简单：从固定路径：当前项目下的 `testfile.txt` 读取整个待编译文件，并将其以字符流的形式存入 `String` 中，以供后续的编译流程使用。设计成单独模块的主要考虑是保持程序各部分的解耦合性。



在 Java 中，相对路径 "`testfile.txt`" 默认该文件在当前项目根目录下。

5.1.2 主要接口

- `extractSourceCode`：从固定路径 `path` 中读取整个文件，利用 `Files.readString(Paths.get(path))` 方法将其以字符流的形式存入一个 `String` 变量中并返回。

5.2 `Lexer` 词法分析

5.2.1 编码器前设计

在项目结构方面，词法分析属于前端内容，将其置于 `src/frontend` 下，为后续架构预留空间。

整体上，词法分析提供对输入文件的解析，最终生成一个 `tokenList` 对象并包含必要信息，为后续语法分析等工作做好准备，`tokenList` 事实上封装了一个 `ArrayList<Token>` 容器。

5.2.2 功能介绍

将预处理输出的字符流，按照 `SysY` 的词法要求，解析转化为 `Token` 流。一个 `Token` 包含三元信息：类型助记符 `TokenType`、值 `String` 以及在源文件所处的行号 `int`。

5.2.3 主要类及接口

5.2.3.1 TokenType

枚举类型，根据词法列出所有可能的单词类型，有如下种类：

▼ 所有单词类型 C#

```
1      // 保留字
2      CONST("CONSTTK"),
3      MAIN("MAINTK"),
4      IF("IFTK"),
5      INT("INTTK"),
6      CHAR("CHARTK"),
7      VOID("VOIDTK"),
8      ELSE("ELSETK"),
9      FOR("FORTK"),
10     CONTINUE("CONTINUETK"),
11     BREAK("BREAKTK"),
12     RETURN("RETURNTK"),
13     GETINT("GETINTTK"),
14     GETCHAR("GETCHARTK"),
15     PRINTF("PRINTF TK"),
16     // 标识符
17     IDSY("IDENFR"),
18     // 字面值
19     INTSY("INTCON"),
20     CHARSY("CHRCON"),
21     STRINGSY("STRCON"),
22     // 运算符
23     PLUSSY("PLUS"),
24     MINUSSY("MINU"),
25     MULTSY("MULT"),
26     DIVISY("DIV"),
27     MODSY("MOD"),
28     ANDSY("AND"),
29     ORSY("OR"),
30     NOTSY("NOT"),
31     LESSSY("LSS"),
32     LESSEQSY("LEQ"),
33     GREATERSY("GRE"),
34     GREATEREQSY("GEQ"),
35     EQSY("EQL"),
36     NOTEQSY("NEQ"),
37     // 分隔符
38     LPARENSY("LPARENT"),
39     RPARENSY("RPARENT"),
40     LBRACKSY("LBRACK"),
41     RBRACKSY("RBRACK"),
42     LBRACESY("LBRACE"),
43     RBRACESY("RBRACE"),
44     COMMASY("COMMA"),
45     SEMISY("SEMICN"),
```

```

46     ASSIGNSY("ASSIGN"),
47     // 无法识别的单词
48     ERROR("ERROR"),;

```

- 提供 `toString` 方法便于打印相关信息以供调试。

5.2.3.2 Token

主要有三个成员变量，完整的表述一个单词，词法分析的最终结果就是由一系列的 `Token` 构成的容器。

▼ 单词组成 Java

```

1     private TokenType type;
2     private String value;
3     private int lineNum;

```

- 提供 `toString` 方法便于打印相关信息以供调试（为契合实验要求，可以仅仅打印二元信息），并重写了 `equal` 与 `hashCode` 方法。

5.2.3.3 ReservedWordsChecker

存储保留字表。识别输入的字符串是否是一个保留关键字，如果是，将其对应的 `TokenType` 返回。

- `isReservedWord`：判断是否是保留关键字。
- `getTokenType`：将某个保留关键字的 `TokenType` 返回。

5.2.3.4 Lexer

词法分析器。从字符流中逐个读取字符，将其识别为某个 `Token`，并加入成员变量 `tokenList` 中，如果无法识别，则转到错误处理部分并输出对应的错误信息。还支持以三元组或者实验要求形式逐个打印各个 `Token` 的具体信息。

- `lexicalAnalysis`：从字符流中逐个读取字符并将其识别成单词，如果无法识别，调用 `ErrorChecker` 添加对应的错误信息。为了保证后续过程正常进行，仍然将错误的符号改为正确的，然后加入 `tokenList`。



主要需要识别的类别有：

标识符或保留字、整数字面值、相等符号或赋值符号、+、-、*、%、单个&（规定要处理的错误）或&&、单个|（规定要处理的错误）或||、<或≤、>或≥、!或≠、三种左右括号、逗号、分号、两种注释或除号、字符面值、字符串面值。

其中字符面值要特别注意转义字符的处理，转义字符转为 `String` 后由于 `\` 符号的存在较一般字符长度大 1。

- `nextToken` 与 `rollback`：取出下一个 `token` 以及回退游标，为语法分析器提供接口。

这里详细阐述如何识别字符字面值和两种注释：

- **字符面值**：当游标 `i` 读取到字符 `'` 时，说明开始出现字符面值，此时游标再读取一位，如果是字符 `\`，说明是转义字符——游标再读取一位，获取转义字符的具体内容，将其存储，然后移动至标识字符面值结束的 `'` 字符；否则是普通字符——存储当前字符（普通字符的内容），然后移动至标识字符面值结束的 `'` 字符。

- **注释：**当游标 `i` 读取到字符 `/` 时，说明出现两种注释或者除号。游标再读取一位，如果不是字符 `/` 或者字符 `*` 说明是除号；如果是字符 `/`，说明是单行注释，移动游标直到读到换行符或者文件结束；如果是字符 `*`，说明是多行注释，移动游标直到当前字符是 `*` 且下一个字符是 `/`。在跳过注释内容时，注意读到换行符仍需使行号加 1。

5.3 Parser 语法分析

5.3.1 编码前设计

在项目结构方面，语法分析也属于前端内容，将其置于 `src/frontend` 下，为后续架构预留空间。设置 `ast` 包以容纳构建需要的语法树，分别对应文法定义说明文档中的函数、变量/常量声明、语句和表达式等等成分，确保覆盖全面，同时尽量简洁高效。

5.3.2 功能介绍

将词法分析器输出的字符流解析成具有层次结构的语法树，本设计采用的语法数据结构将在下面的语法树结构介绍中详细说明。为了兼顾实验的输出格式要求和后续分析的便利，本设计将解析构建语法树的过程和输出要求信息的过程完全分离——先完全构建好整个语法树，再统一输出。同时，在解析构建时对实验文法的各个语法成分进行优化，以减小代码规模，降低冗余度；在输出时复原所有非必要的成分以符合实验要求。

5.3.3 语法树结构

首先对实验提供的文法进行优化，包括：增加中间表示、简化合并、消除递归等，得到的解析用等价文法见下图。

补充说明：

- `(Const)Decl` 的意思是：用一个类（可以命名成 `Decl`）统一代表原来的 `ConstDecl` 和 `VarDecl`，然后在内部用一个变量来标识其是否为 `Const` 的。`(Main)FuncDef` 等表示同理。
- 用单引号标注的成分均为终结符，即单词。其中 `id` 是标识符，`strcon` 是字符串字面量，`intcon` 是整数字面量，`charcon` 是字符字面量。

▼ 等价文法 Java

```

1  编译单元 CompUnit → {(Const)Decl} {(Main)FuncDef} (Main)FuncDef
2  -----
3  声明 (Const)Decl → ('const') 'int'或者'char' (Const)Def { ',' (Const)Def } ';'
4
5  标识符定义 (Const)Def → 'id' [ '[' AddExp ']' ] '=' (Const)InitVal
6
7  初值 (Const)InitVal → AddExp | '{' [ AddExp { ',' AddExp } ] '}' | 'strcon'
8
9  加减表达式 AddExp → MulExp {('+' | '-') MulExp}
10
11  乘除模表达式 MulExp → UnaryExp {('*' | '/' | '%') UnaryExp}
12
13  一元表达式 UnaryExp → {('+' | '-' | '!')} PrimaryExp | FuncCall
14
15
```

```

16 函数调用 FuncCall → 'id' '(' [AddExp { ',', AddExp } ] ')'
17
18 基本表达式 PrimaryExp → '(' AddExp ')' | LVal | 'intcon' | 'charcon'
19
20 左值表达式 LVal → 'id' '[' [ AddExp ] ']'
21 -----
22 函数定义 (Main)FuncDef → 'void'或者'int'或者'char' 'id'
23 '(' [FuncFParam { ',', FuncFParam } ] ')' Block //非main函数
24 | 'int' 'main' '(' ')' Block // main函数
25
26 函数形参 FuncFParam → 'int'或者'char' 'id' '[' ']'
27
28 语句块 Block → '{' { BlockItem } '}'
29
30 语句块项 BlockItem → (Const)Decl | Stmt
31
32 语句 Stmt → AssignStmt
33 | IfStmt
34 | ForStmt
35 | PrintfStmt
36 | [AddExp] ';' //有无Exp两种情况
37 | Block
38 | 'break' ';'
39 | 'continue' ';'
40 | 'return' [AddExp] ';' // 1.有Exp 2.无Exp
41
42 赋值语句 AssignStmt → LVal '=' AddExp ';'
43 | LVal '=' 'getint' '(' ')' ';'
44 | LVal '=' 'getchar' '(' ')' ';'
45
46 if语句 IfStmt → 'if' '(' LOrExp ')' Stmt [ 'else' Stmt ]
47
48 逻辑或表达式 LOrExp → LAndExp { '||' LAndExp }
49
50 逻辑与表达式 LAndExp → EqExp { '&&' EqExp }
51
52 相等性表达式 EqExp → RelExp { ('==' | '!=') RelExp }
53
54 关系表达式 RelExp → AddExp { ('<' | '>' | '≤' | '≥') AddExp }
55
56 for语句 ForStmt → 'for' '(' [LVal '=' AddExp] ';' [LOrExp] ';'
57 [LVal '=' AddExp] ')' Stmt
58

```

根据上面的等价文法，可以很方便地直接按各个非终结符成分构建类——每个非终结符类包含若干个非终结符子成员类和终结符 `Token` 类型成员变量。进而组成整个语法树，存放于 `parser.ast` 包下。

5.3.4 主要类和接口——`Parser` 和 `parse` 方法

语法分析器使用一个 `CompUnit` 类型的静态成员变量，存储解析后整个编译单元的结构和成分。核心处理机制是对 `CompUnit` 进行解析的 `parse` 方法。主要应用了递归下降和超前读取。

- **递归下降**：解析时，将解析任务不断下降派发给各个子成分，直至到达不可再分的子成分为止（输出时也同样递归下降地打印各个子成分）。比如最顶层的 `parse` 方法，解析整个 `CompUnit`，可以递归下降为 `parse` 若干个 `Decl` 和 `FuncDef`。本设计在上述语法树结构的基础上，为每个语法结构类都设置并实现了 `parse` 成员方法。
- **超前读取**：为了区分到底使用一个语法成分多个右部中的哪一个，需要超前读取 `token` 来进行判别。比如 `Block` 语法成分的第一个 `token` 一定是左大括号 `{`。当然，超前读取后需要恢复到原来的状态。这就要求我们的 `Lexer` 在取出 `token` 时还要拥有回退功能，实际上就是将读取的游标减 1 表示回退一步。特别注意，在处理语法分析要求处理的错误时，如缺少右小括号，如果 `Lexer` 读取的下一个字符不是预期的右小括号，在报错的同时也要使 `Lexer` 回退一步，以保证下一个语法结构解析时的完整性。

6 中端设计



中端的主要任务是：接受前端输出的抽象语法树作为输入，对其进行语义分析，并为其生成中间代码。

6.1 Visitor 语义分析

6.1.1 编码前设计

在项目结构方面，语义分析属于中端内容，将其置于 `src/middle` 下，为后续架构预留空间。

整体上，语法分析进行最后的错误检查，并且构建出符号表辅助下一步中间代码的生成。

6.1.2 功能介绍

遍历语法分析器输出的语法树，对其中的所有变量以及函数构建符号表，并检查多种类型的错误，最终确认输入程序的正确性。错误的输入程序将不再参与后续的编译流程。

6.1.3 主要类

6.1.3.1 SymbolTable

符号表是进行语义分析的基础数据结构。符号表的基本单位是符号 `Symbol`。在本设计中，符号表分别装载两种类型的符号：变量符号 `VarSymbol` 以及函数符号 `FuncSymbol`，将每个具名变量以及函数均记录为一个符号。设计符号的关键在于需要明确每种符号应该记录哪些内容，其既需要满足语义分析的要求——输出当前符号所在的作用域编号，又要为后续的中间代码生成提供相应数据。基于这一分析标准，本设计对两种符号分别设置了如下所示的成员：

- 变量符号：

▼ `VarSymbol`

Java

```
1 // 语义分析相关
2 // 变量名
3 private String name;
4 // 是否有const修饰
```



```

5     private boolean isConst;
6     // 是否是全局变量
7     private boolean isGlobal;
8     // int、char、intArray或者charArray
9     private Type type;
10    // 中间代码相关 (LLVM IR)
11    // 初始值
12    private Value initValue;
13    // 对应的变量
14    private Value var;
15    // 如果是函数参数，记录其位序
16    private int paramIndex;
17    // 如果是函数参数，记录其类型
18    private ValueType paramType;

```

- 函数符号：

```

▼ FuncSymbol Java
1     // 语义分析相关
2     private String name;
3     private ArrayList<VarSymbol.Type> paramTypes;
4     private Type returnType;
5     // 中间代码相关 (LLVM IR)
6     private ArrayList<ValueType> paramValueTypes;
7     private int paramNum;
8     private Value returnValue;

```

6.1.3.2 Visitor

语义分析器。存储语法分析器生成的 `CompUnit` 类型的静态成员变量，对其进行遍历。需要解决的核心问题是错误检查和符号表构建。

- 递归下降的 `visit` 错误检查：参照语法分析器的思路，同样地递归下降处理各个语法成分。本设计将语法分析的 `class.parse()` 形式改为 `visit(class)` 形式，使不同成分的 `visit` 方法之间能在同一个类 (`Visitor`) 下，从而更加灵活的传递需要的数据。
- 符号表构建：本设计分别存储变量符号与函数符号，共同组成符号表数据结构。符号应该至少记录对象的类型和名字。此外，每张符号表记录 `father` 成员变量，指向自己的外层符号表。在 `Visitor` 遍历过程中，首先创建全局符号表，并记录当前符号表为全局符号表。只有当遍历进入 `Block` 成分时：用临时变量记录当前符号表、新建符号表让其指向当前符号表、将当前符号表改为新符号表。只有当退出 `Block` 成分的遍历时，取出当前符号表的外层符号表、将当前符号表改为外层符号表。对于实验要求的符号表编号，可以为 `Visit` 类设置静态计数成员变量，新建符号表时，将其值作为符号表的编号，然后使其加 1 即可。

6.2 LLVM IR 中间代码生成

6.2.1 编码前设计

在项目结构方面，中间代码生成也属于中端内容，将其置于 `src/middle` 下，设置 `llvm` 包以容纳构建需要的 `LLVM IR` 数据结构。详见下面的介绍。

6.2.2 功能介绍

在语义分析完成之后，我们得到的是没有任何错误的抽象语法树，理论上下一步就可以将这棵语法树翻译成对应的目标代码。但是，目标代码往往是与具体机器相关的底层语言，如本设计采用的 MIPS32，而抽象语法树本质上还是由源语言构建的，往往是一种高级语言。可以看到，将一种高级语言直接翻译成机器语言，跨度很大而且目标语言依赖于机器的特性也使得编译器的可扩展性不强。因此我们引入一个过渡环节——先将抽象语法树先翻译成一种不依赖具体机器的中间代码，再由中间代码翻译成目标代码。这样能有效降低每个编译步骤的难度梯度，也提高了编译器的可扩展性。

对于中间代码的选择，本设计采用 LLVM IR 中间表示。这种表示有体系化的语法格式和成熟的工具链支持，能够很方便地验证其正确性。为了将语法树翻译成这种表示，我们需要建立相关的数据结构，然后编写相应的构建方法。下面将分别着重介绍这两个部分。

6.2.3 LLVM IR 数据结构

LLVM IR 可以被看作是一种编程语言，它有自己的完整语法规则。实验要求的源代码结构并不复杂，因此无需一字不差地照搬整个 LLVM IR 的体系，这显然也是难以做到的。我们需要在借鉴其思想的基础上，构建一套属于自己的，简洁又正确的数据结构。

在本设计中，将 LLVM IR 自顶向下划分成 module 模块、globalvar 全局变量和 function 函数、basicblock 基本块、instruction 指令以及最后的 item 项五层结构。前四层也是 LLVM IR 标准结构所使用的划分层次，最后一层则是本设计自行添加的辅助层次。此外，所有层次的所有成分都继承自 Value 类，每一个 Value 有其对应的 ValueType，ValueType 也派生诸多子类。基于此我们也可以将整个 LLVM IR 看作类型系统和值系统的结合。下面先介绍本设计采用的类型系统，再将按照层次划分详细介绍各个 Value，并指出每个成分对应的 ValueType。

6.2.3.1 类型系统

- `UnknownType`：未知类型，可以理解为一种占位性质的类型。不包含更多信息，可以使用单例模式。
- `FuntionType`：函数 Value 的类型。该类型包含函数的形参个数、形参类型以及返回值类型信息。
- `LabelType`：基本块 Value 的类型。不包含更多信息，可以使用单例模式。
- `PointerType`：指定所引用的类型，引用类型仍然可以是 `PointerType`。
- `IntegerType`：指定位宽。支持 i32 整数、i8 字符以及 i1 布尔三种不同位宽的类型。
- `ArrayType`：指定数组的元素类型和数组的大小。元素类型仍然可以是 `ArrayType`。
- `VoidType`：函数返回值为 `void`，`void` 也看做是一个 Value，其类型为 `VoidType`。不包含更多信息，可以使用单例模式。

6.2.3.2 模块

一个模块包含若干个全局变量和函数。通俗来讲模块的涵盖范围相当于一个文件。以 Java 语言为例，这里的一个模块就对应其一个类、接口或者是枚举。由于实验仅要求将一个源代码文件编译为目标代码，因此在实际构建过程中只需要一个模块实例。

模块的类型不是我们关注的重点，可以将其指定为 `UnknownType`。

6.2.3.3 全局变量与函数

全局变量对应 `SysV` 文法中的在 `MainFuncDef` 和 `FuncDef` 之外的所有 `Decl`。函数则是对应剩下的 `MainFuncDef` 和 `FuncDef`。

一个函数包含若干个基本块。

全局变量的类型是 `ArrayType` 或者是 `IntegerType`。

函数的类型是 `FunctionType`。

6.2.3.4 基本块

一个基本块包含若干个指令。基本块只有最后一条指令能是且必须是跳转到其他基本块的跳转指令。基本块可以指定 `label` 标签名，详见命名约定一节。

基本块的类型是 `LabelType`。

6.2.3.5 指令

一个指令包含若干个项（可以理解成操作数）。指令的具体要求参见 `LLVM IR` 的规定。需要注意的是，我们无需实现所有的指令，并且可以根据自己的需要将多种结构相同的指令合并为一种，在类内加以区分，让整个实现更加简洁。例如在本设计中，实现了二元运算指令类 `BinaryInst` 包含了加减乘除以及取模五种运算。

指令的类型应该指定为其返回结果的类型或者 `VoidType`（这里的类型不是代表指令本身是哪种指令）。

6.2.3.6 项

项细分为 `literal` 字面量和 `localvar` 本地变量。每个本地变量实际上对应一个虚拟寄存器，其命名约定见下节。

字面量的类型是 `IntegerType`。本地变量的类型指定与全局变量相同。

6.2.3.7 命名约定

`LLVM IR` 对于虚拟寄存器的命名规则有详细规定，如果单纯使用数字需要保证严格递增。为了减少不必要的麻烦，本设计采用字母和数字混合命名。具体如下：

- 形参命名：`p0`、`p1`.....每个函数重置。
- 基本块命名：`entry`、`lb0`、`lb1`.....每个函数重置。
- 本地变量命名：`lv0`、`lv1`.....每个函数重置。

6.2.4 主要类和接口— `Builder` 和 `build` 方法

构建中间代码的总体思路是对于语法树的每种成分，为其编写针对性的 `build` 方法。下面重点介绍实现难度较高的几类构建情况。

6.2.4.1 函数参数装载

在 `LLVM IR` 中，直接 `use`（引用）函数形参会出现报错：未经定义的使用。因此我们需要一个对形参的装载过程——将形参的值 `store` 到本地开辟的内存空间中。再利用虚拟寄存器进行引用。对于每个有参数的函数，统一在函数体开头设置第一个基本块 `entry` 进行此工作。`entry` 的最后一条 `instruction` 跳转到下一个紧邻的基本块，开始构建对应函数的第一条语句。

一个可行的参考示例如下所示：

```
C-like
1  int func_with_param(int a, char b, int arr[], char str[]) {
2      ...
3  }
4
5  define dso_local i32 @func_with_param(i32 %p0, i8 %p1, i32* %p2, i8* %p3) {
6  entry:
7      %lv0 = alloca i32
8      store i32 %p0, i32* %lv0
9      %lv1 = alloca i8
10     store i8 %p1, i8* %lv1
11     %lv2 = alloca i32*
12     store i32* %p2, i32** %lv2
13     %lv3 = load i32*, i32** %lv2
14     %lv4 = alloca i8*
15     store i8* %p3, i8** %lv4
16     %lv5 = load i8*, i8** %lv4
17     br label %lb0
18 lb0:
19     ...
20 }
```

6.2.4.2 数组元素位置计算

LLVM IR 的数组元素位置计算需要使用 `gep` 指令。计算出某元素的地址后再对对应的值进行操作，或者是计算出第 0 个数组元素的地址做为整个数组的起始地址，传递给 `i32*` 类型的形参。

`gep` 指令所需要的 `index` 可能会让人难以理解。到底要用多少个索引，这些索引都代表何种含义？这里用一个实例来说明：

```
6502 Assembly
1  %lv0 = getelementptr inbounds [3 x i32], [3 x i32]* @constIntArray, i32 0, i32 0
```

注意到这条指令计算的是 `constIntArray[0]` 的地址。但是在索引中出现了两个相同的偏移量 `i32 0`。这两个偏移量实际上有着不同的含义：偏移量是针对前一项的基指针而言的。在此例中基指针是**数组指针类型**的 `constIntArray`。数组指针指向数组的起始地址，而数组起始地址又指向数组首元素，因此用 C 语言的类型系统解释，该基指针可以说是 `int**` 类型。第一个偏移量 `i32 0` 就可以看作是：

```
C
1  int** p = constIntArray;
2  p += 0;
```

`p` 指向一个 `int*` 的变量，因此其移动的步长（1 偏移量）等于 `int*` 类型的大小。我们想要取到正好位于该地址的数组起始地址值，显然这个偏移量应该是 0。

那么第二个偏移量的含义是：

C

```

1  int* pp = *p;
2  pp += 0;

```

此时我们拿到了数组的起始地址，要想找到第 0 个元素，还需要第二个偏移量。`pp` 指向 `int` 类型的变量，步长是 `int` 类型的大小，这个偏移量事实上对应的就是数组索引 `constIntArray[0]` 中的 0。如果我们需要取第 3 个元素地址，那么显然第二个偏移量应该改为 `i32 3`。

6.2.4.3 算术表达式

- 一元表达式：`LLVM IR` 的指令是统一的二元运算指令，`SysY` 的 `unaryExp` 一元运算需要等价地翻译成二元运算。对于 `+` 运算可以直接忽略；对于 `-` 运算，可以添加左操作数为字面量 0 构成二元 `sub` 运算。对于 `!` 运算首先理解其含义——`!unaryExp` 求值应该先计算 `unaryExp`，若结果为 0，整个表达式结果为 1，否则整个表达式结果为 0。基于此，我们可以使用 `icmp` 指令，`icmp i32 eq unaryExp, 0` 表示 `unaryExp` 与 `0` 做等于比较，结果为真返回 1。这也就达到了逻辑非的运算效果。
- 多元表达式：如果对 `SysY` 文法 `AddExp` 成分推导消除递归，实际上代表多元加减运算，如 `1 + 2 - 3 + 4`。翻译成二元运算应该从左往右计算：`lv1 = 1 + 2 ; lv2 = lv1 - 3 ; lv3 = lv2 + 4`。核心是将前一步运算的结果做为下一步的操作数。

6.2.4.4 条件跳转与短路求值

`SysY` 文法要求对条件表达式进行**短路求值**，因此对于一个条件表达式不能整体地计算，而是每次计算一个 `eqExp`，然后做相应的判断，比如下面这个实例：

```

1  if( a && b || c && d ){
2  ...} else {
3  ...}

```

JavaScript

对于这个 `if` 语句，正确的 `LLVM IR` 翻译思路应该是：

- 当前块：**计算 `a` (`a` 是一个 `eqExp`)，若结果为 0，说明 `a && b` 结果为 0，新建并跳转到下一个 `LAndExp (c && d)` 的计算块；若结果为 1，新建并跳转到下一个 `eqExp (b)` 的计算块。
- `LAndExp (c && d)` 的计算块：**计算 `c` 若结果为 0，说明 `c && d` 结果为 0，此时 `a && b` 结果已经为 0，故整个 `LOrExp` 结果为 0，新建并跳转 `else` 块；若结果为 1，新建并跳转到下一个 `eqExp (d)` 的计算块。
- `eqExp (b)` 的计算块：**计算 `b` 若结果为 0，说明 `a && b` 结果为 0，跳转到下一个 `LAndExp (c && d)` 的计算块；若结果为 1，此时 `a` 结果已经为 1，那么 `a && b` 结果为 1，进而整个 `LOrExp` 结果为 1，新建并跳转到 `then` 块。
- `eqExp (d)` 的计算块：**计算 `d` 若结果为 0，说明 `c && d` 结果为 0，此时 `a && b` 结果已经为 0，进而整个 `LOrExp` 结果为 0，跳转到 `else` 块；若结果为 1，此时 `c` 的结果已经为 1，那么 `c && d` 结果为 1，进而整个 `LOrExp` 结果为 1，跳转到 `then` 块。
- `then` 和 `else` 块：**继续向后读取，根据内容进行 build。

6.3 LLVM IR 中间代码优化

6.3.1 公共子表达式删除优化

如果表达式 `x op y` 先前已被计算过，并且从先前的计算到现在，`x op y` 中变量的值没有改变，那么 `x op y` 的这次出现就称为公共子表达式。

用一个比较简单的例子说明，考虑到下列代码：

```
a = b * c + g;
d = b * c + e;
```

C

可以观察到 `b * c` 是两项表达式中的公共子表达式。如果计算这个子表达式并将其计算结果存储起来的开销，低于重复计算这个子表达式的开销，则能够将以上代码转换成以下代码：

```
temp = b * c;
a = temp + g;
d = temp + e;
```

C

因此对于如下的程序：

```
#include <stdio.h>
int main(){
    int x, y, z;
    z = (x + y) * (x + y);
    printf("%d", z);
    return 0;
}
```

C

对于编译器来说 `z = (x + y) * (x + y)` 这句话经过公共子表达式消除后会变成：

```
tmp = x + y
z = tmp * tmp
```

Python

6.3.2 死代码删除

死代码是指程序中不会被执行到的代码，但仍然存在于程序中，浪费了空间和资源。为了提高程序的性能和可维护性，需要进行死代码消除。

通过分析程序的控制流图和数据流图，可以确定哪些代码是死代码，并将其从生成的代码中删除。死代码消除通常是与其他优化技术结合使用的，例如常量传播、复写传播和循环优化等。这些技术可以进一步减少生成的代码中的死代码数量，提高程序的性能和效率。

在本设计的场景中，无法进入的 `if-then` 代码块、`if-else` 代码块、`for` 循环语句块，或者 `return` 语句之后的所有语句，或被视为死代码，在中间代码生成阶段需要将其删除。

7 后端设计



后端的主要任务是：接受中端输出的 `LLVM IR` 中间代码作为输入，将其翻译成目标代码 `MIPS32`，然后对 `MIPS32` 进行一定程度的优化。

7.1 目标代码生成

7.1.1 编码前设计

在项目结构方面，目标代码生成属于后端内容，将其置于 `src/backend` 下，设置 `mips` 包以容纳翻译需要的 `MIPS32` 数据结构。主要是一系列常用的 `MIPS32` 指令。

7.1.2 功能介绍

我们在前面的环节中已经得到了正确的 `LLVM IR` 中间表示，现在需要将其翻译为 `MIPS32` 指令序列。两者语法在总体上比较接近。可以采取读取一条指令翻译一条的策略，生成最终的目标代码文件。

7.1.3 主要类和接口— `Translator` 和 `translate` 方法

本设计参照构建中间代码的思路，对于每个 `LLVM IR` 的 `instruction`，为其编写针对性的 `translate` 方法。下面重点介绍本设计针对一些情况的特殊处理思路。

7.1.3.1 统一类型

`SysV` 文法中，基础数据类型有 `int` 和 `char` 两种，由于 `MIPS32` 要求所有数据必须 4 字节对齐，对于 1 字节大小的 `char` 类型变量存取需要格外注意。为了减少不必要的麻烦，本设计采用将所有基础类型统一提升为 `int` 类型的策略。该策略带来以下几个方面的便利：

- **统一的存取**：在向内存中存储或从内存中加载变量时，只需要使用 `sw` 和 `lw` 两种指令，不必再生成 `sb` 和 `lb` 指令对 `char` 变量进行操作。
- **统一的偏移量计算**：所有数组均为 `int` 类型数组。每个元素相对于数组首地址的偏移量（字节数）均为索引值乘以 4。

在使用该策略时，我们需要对 `int` 和 `char` 变量之间的类型转换语句进行相应处理。对应到 `LLVM IR` 中，表现为对 `zext` 指令和 `trunc` 指令的处理。`zext` 是类型扩展，由于已经进行了统一提升，这部分工作自然完成。`trunc` 是类型压缩，对于将一个 `int` 变量压缩为 `char` 变量的操作，实际上是取其低 8 位的结果。因此我们应该使用 `MIPS32` 的 `andi` 指令，让 `int` 型变量和立即数 `b0000 0000 1111 1111` 进行按位与运算得到低 8 位的结果。

7.1.3.2 寄存器分配策略

在本设计中，由 `RegisterFile` 统一调度所有的寄存器分配。本设计采取的寄存器分配策略具体规则如下：

- 将 `$t0-$t9` 和 `$s0-$s7` 视为 18 个相同的通用寄存器进行统一分配。建立索引循环遍历这些寄存器，以期实现较为均衡的分配。
- 对于函数参数，优先装载至 `$a0-$a3` 四个寄存器，多余的参数分配到栈上。对于函数返回值，统一存储至 `$v1` 寄存器。

- 对于本地变量（虚拟寄存器），在 LLVM IR 中通过 `alloca` 指令开辟空间的变量，直接分配到栈上；其余的本地变量优先考虑为其分配物理寄存器。此外，仅仅为非指针类型的虚拟寄存器分配相应的物理寄存器，指针类型的虚拟寄存器通过增加特殊字段存储其与对应值寄存器的关联关系。当物理寄存器不足时，分配到栈上。
- 在每个虚拟寄存器首次使用时尝试为其分配物理寄存器，在其最后一次使用时立即将对应的物理寄存器回收。

7.1.3.3 栈管理策略

使用 `$fp` 寄存器管理栈空间。`$fp` 的值在整个翻译过程中宏观上保持不变。通过每次递增 4 的偏移量在栈上不断压入变量。每个变量记录其相对于 `$fp` 的偏移量以保证后续能被准确访问。由于基地址固定，不必担心翻译顺序和实际代码执行顺序不一致导致变量存储位置错乱的问题。

7.1.3.4 函数调用

每次进行函数调用时应该进行以下步骤：

- **保存寄存器**：采用最保守的策略，保存（压入栈中）调用者函数的 `$ra` 寄存器、所使用的所有参数寄存器以及生存周期横跨该调用语句位置的虚拟寄存器所关联的所有物理寄存器（最多 18 个）。
- **装载参数**：优先装载至 `$a0-$a3` 四个寄存器，多余的参数分配到栈上。特别注意，由于下一步的 `$fp` 临时移动，分配到栈上的位置应该再加上下一步的 `$fp` 临时移动偏移量。
- **开辟栈帧**：在每次调用函数时，应该为其开辟独立的栈帧空间，避免自身递归调用时出现空间重叠而丢失上一级调用的内存信息。通过使基指针 `$fp` 在调用前加上固定值 `offset`，在调用结束时还原可以实现这一点。仅改变基指针而无需改变所有记录的相对基指针的偏移量使得翻译程序的修改量达到最小。



具体开辟多大空间的栈帧需要经过一定的实践。空间太小仍然会导致内存重叠，而空间太大会使得栈总空间过大，也会产生不可预知的问题。在本次实验中，开辟 `0x550` 字节大小的空间是一个合理的数值。

- `jal` 指令调用函数，跳转。
- **恢复寄存器**：将上述保存的所有寄存器值弹出栈，返回相应的寄存器中。

7.1.3.5 指令选择

对于加减指令，为了防止大数相加减时出现溢出情况，请务必选择无符号运算的版本，即 `addu` 和 `subu`。如果使用了对应的立即数伪指令，也需要使用相应的无符号版本，即 `addiu` 和 `subiu`。

7.2 目标代码优化

7.2.1 编码前设计

目标代码优化属于可选内容，为了减少遍历的 `pass`，将其集成实现在 `src/backend/Translator` 下，将中间表示翻译为目标代码时就决定是否进行优化，设置相应的优化开关以便翻译器能输出未优化和优化后的两份目标代码。同时，根据实验的评判标准，优先优化对 `cycle` 统计贡献最大的指令。

7.2.2 功能介绍

我们在前面的环节中已经得到了正确的 `MIPS32` 目标代码，现在需要尽可能使其执行效率提高。采用各种优化方法，或减少指令条数，或将复杂指令转化为等价的简单指令，最后得到既保证正确，又大大提高执行效率的目标代码。

7.2.3 主要优化策略

7.2.3.1 窥孔优化

Peephole optimizations（窥孔优化）是编译器优化中的一种技术，它基于一些特定的编码模式或者汇编代码序列，针对代码的局部片段进行优化。具体来说，它是一种局部优化，针对一个固定长度的代码片段，通常是 3 到 10 条指令，来尝试优化这些指令的执行效率和代码质量。窥孔的主要思想是在编译器生成的代码中，通过扫描一段代码中连续的几条指令，寻找其中可能存在的性能问题或者可以优化的代码段，然后对这些代码进行变换，以达到更高的执行效率或者更简洁的代码结构。

- 删除无用的指令：如果一些指令的执行结果不会被用到，那么这些指令可以被删除掉，从而减少代码的运行时间和代码长度。
- 简化控制流：例如，将条件跳转语句转换为条件赋值语句，可以简化代码结构，提高代码的执行速度。
- 简化指令序列：例如，将多个相同的指令序列合并为一个，可以减少指令的数量，提高代码的执行效率。

下面举例说明本设计中窥孔优化的应用：

对于 `LLVM IR` 表示中的 `entry` 块，该块在本设计中的作用是参数装载，而在 `MIPS32` 中该步骤应该在函数跳转前就完成，因此该块可不必翻译，只保留该块标签和跳转指令即可，其他无用指令均可以删除。

此外，观察发现该块的跳转指令是跳转到下一个相邻标签的，因此也可以删除，最终仅保留本块标签。

7.2.3.2 寄存器分配优化

可以采用线性扫描算法优化寄存器的分配策略，算法伪代码思想参考如下：

```
Rust
1 LinearScanRegisterAllocation:
2     active := {}
3     for i in live interval in order of increasing start point
4         ExpireOldIntervals(i)
5         if length(active) == R
6             SpillAtInterval(i)
7         else
8             register[i] := a register removed from pool of free registers
9             add i to active, sorted by increasing end point
10    ExpireOldInterval(i)
11    for interval j in active, in order of increasing end point
12        if endpoint[j] >= startpoint[i]
13            return
14        remove j from active
15        add register[j] to pool of free registers
16    SpillAtInterval(i)
```

```

17     spill := last interval in active
18     if endpoint[spill] > endpoint[i]
19         register[i] := register[spill]
20         location[spill] := new stack location
21         remove spill from active
22         add i to active, sorted by increasing end point
23     else
24         location[i] := new stack location

```

live interval 是变量的生命期，用活跃变量分析可以计算得到。活跃变量分析在理论课中已经介绍，此处不再赘述。

7.2.3.3 乘法运算优化

- 左操作数和右操作数均为立即数时，直接计算出结果，使用 `li` 伪指令加载即可（常数折叠）。
- 左操作数和右操作数有一个是立即数，且立即数为 2 的倍数，将乘法指令优化为 `sll` 移位指令。注意如果立即数是负数，可以将其取绝对值，当作正值优化，最后多生成一条被减数为 0 的 `sub` 指令。
- 其余情况不优化。

7.2.3.4 除法运算优化

- 左操作数和右操作数均为立即数时，直接计算出结果，使用 `li` 伪指令加载即可（常数折叠）。
- 只有右操作数是立即数时：

立即数为 2 的倍数，将除法指令优化为 `sra` 移位指令。注意如果整体表达式为负，被除数需要加上 `padding` 值再做移位，`padding` 值等于除数绝对值减去 1。因为 MIPS32 有符号除法中负数结果是向 0 取整，而直接移位是向更小的整数取整，两者计算结果会有偏差。

立即数不是 2 的倍数，可以将其转为乘法和移位指令。同样注意如果整体表达式为负，最后的结果需要加 1。优化方法参考教程的如下内容：

除法优化的思路是将除法指令转化为乘法指令和移位指令，即如下公式：

$$quotient = dividend \div divisor = (dividend * multiplier) \gg shift$$

该公式先乘一个较大的常数，然后用右移 `shift` 位得到最终的答案，使用该式子计算除法的核心是如何得到 `multiplier`，另外由于 MIPS 架构是 32 位的，还必须要考虑溢出的问题。MIPS 在执行乘法的时候会将结果保存到 hi 和 lo 寄存器当中，在被除数乘 `multiplier` 之后，很容易就会超过 32 位，这个就会出现一部分答案在 hi 寄存器中，一部分答案在 lo 寄存器当中，所以我们需要使 `multiplier` 尽量大（超过 2^{32} ），能够使答案的部分在 hi 寄存器当中。所以最终的公式可以写为如下形式

$$quotient = \lfloor n \div d \rfloor = \lfloor m * n \div 2^{N+l} \rfloor$$

其中 N 是机器码长度 32，最终获得答案的公式为 `SRL(MULUH( $m, n$ ),  $shift$ )`，`MULUH` 表示乘法之后取 HI 寄存器。

下面附上本设计取得 `multiplier( $m$ )` 和 `shift( $l$ )` 的方法：

```

1 // prec取定值32，就是教程中的推荐求法。
2 public static long[] choose_multiplier(int d, int prec) {

```

Java

```

3      long l = (long) Math.ceil((Math.log(d) / Math.log(2)));
4      long sh = l;
5      long m_low = (long) Math.floor(Math.pow(2, 32 + l) / d);
6      long m_high = (long) Math.floor((Math.pow(2, 32 + l) +
7      Math.pow(2, 32 + l - prec)) / d);
8      while ((Math.floor(m_low >> 1) < Math.floor(m_high >> 1)) && sh > 0) {
9          m_low = (long) Math.floor(m_low >> 1);
10         m_high = (long) Math.floor(m_high >> 1);
11         sh = sh - 1;
12     }
13     return new long[]{m_high, sh};
14 }

```

- 其余情况不优化。

7.2.3.5 取模运算优化

- 左操作数和右操作数均为立即数时，直接计算出结果，使用 `Li` 伪指令加载即可（常数折叠）。
- 只有右操作数是立即数时：

立即数为 1 或者 -1，结果必定为 0，使用 `Li` 即可。

立即数为 2 的倍数且为正数。假设立即数 $imm = 2^n$ ，记 $num = imm - 1$ 。那么原来的除法指令求余数可以被优化成：被除数 `sra` 算术右移 31 位再逻辑右移 $32 - n$ 位得到 `padding`、被除数加上 `padding` 然后与 `num` 做 `andi` 运算得到 `res`、`res` 最后减去 `padding` 得到最终结果。`padding` 的存在是考虑到被除数可能为负值的情况。

- 其余情况不优化。

7.2.3.6 指令选择优化

为了更方便的实现更多功能，以及更加方便翻译高级语言，Mars 给我们提供了充分的伪指令使用，例如 `li $t1, 100`、`move $1, $2`、等，能够缩短指令的条数，增加代码的可读性。

但实际上很多时候伪指令虽然表面上降低了指令的条数，但是实际上反而会使竞速评判标准的 FinalCycle 值增加，在循环次数很多的循环体当中，这一影响格外明显，哪怕每个语句多翻译了一条代码都会严重影响性能。

`subi` 等语句其实是非常常见的计算语句，在循环当中也可能高频出现。因此用基础指令代替 Mars 低效的伪指令，或者自己封装一套翻译机制是提升性能的有效方法。

本设计中，全部使用 `addi` 基础指令来代替 `subi` 伪指令（准确来说是 `addiu` 代替 `subiu`），在开辟函数栈帧空间时对 `$fp` 的固定操作处便可使用这一优化。

7.2.3.7 常数折叠优化

对于 MIPS32 代码中 `add`、`sub`、`muli`、`div`、`seq`、`sne` 等这些二元运算指令，当两个操作数均为立即数时，直接计算出结果，用 `Li` 代替原本的运算指令。